

Fully Homomorphic Encryption (FHE)

Part II: The FHE Engine - LWE Problem and the Noise Agony

Ksawery Moźdzynski

Agenda for Today

1. From ElGamal to True FHE

Limitations of Partial Homomorphism

2. Mathematical Essentials & The LWE Problem

Vectors, Dot Product, and Learning With Errors

3. LWE Encryption & Decryption

The Data Box, Scaling Factor, and Removing the Mask

4. Homomorphic Operations

Addition, Multiplication, and the Noise Explosion

5. Time for Practice!

Introduction to the OpenFHE Library

Recall from Day 1: Historical Homomorphism

The ElGamal Scheme

- Yesterday, we explored the **ElGamal** scheme, which gave us a taste of homomorphic properties.
- It demonstrated that we can perform meaningful operations on encrypted data without needing the private key.
- Specifically, it natively supports the *multiplication* of ciphertexts.

But is this enough to build a secure cloud computer?

The Limitation: Multiplicative Only

! Partial Homomorphism

ElGamal allows multiplying ciphertexts:

$$E(x) \cdot E(y) = E(x \cdot y)$$

But it **cannot** perform addition:

$$E(x) + E(y) = ?$$

Why is this a problem?

To build complex AI models or run any arbitrary software on encrypted data, we need a **Turing Complete** system.

This requires the ability to evaluate **both** addition and multiplication (which together can form any logic gate).

Mathematical Refresher: What is a Vector?

Before we build the FHE Engine, we need vectors.

In simple terms, a **vector** is an ordered list or a 1-dimensional array of numbers. Instead of working with a single number, FHE groups multiple numbers together.

Notation and Example

We typically denote a vector with a small arrow on top, like $\vec{v}$.
An n -dimensional vector contains n individual numbers (elements):

$$\vec{v} = (v_0, v_1, v_2, \dots, v_{n-1})$$

For example, a secret key vector $\vec{s}$ of length 3 could look like this:

$$\vec{s} = (2, -1, 3)$$

Mathematical Refresher: The Dot Product

How do we multiply two vectors together?

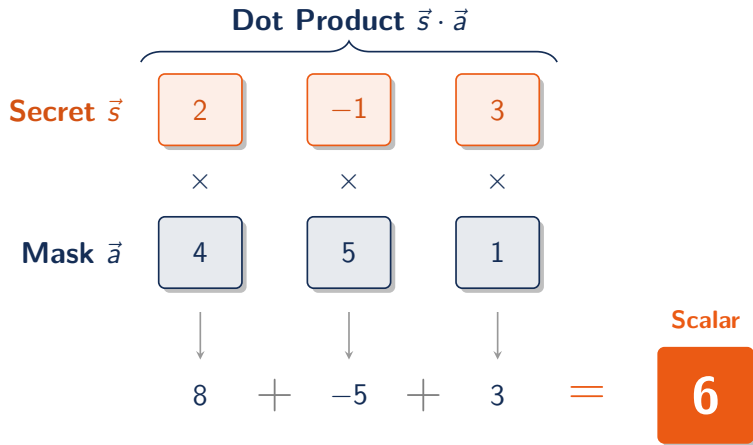
The **Dot Product** (iloczyn skalarny) is the core mathematical operation of modern Fully Homomorphic Encryption.

How it works

It takes two vectors of the **exact same length**, multiplies their corresponding elements together, and adds the results to produce a **single scalar number** (not a vector!).

Let's see how this looks step-by-step on the next slide...

Visualizing the Dot Product



The Foundation: Standard Linear Equations

Solving standard equations is easy.

Given enough equations, finding the exact secret values (s_1, s_2) is a trivial task for any modern computer (e.g., using Gaussian elimination).

✓ Standard Linear Equations

$$14 = 4s_1 + 3s_2$$

$$9 = 2s_1 + 1s_2$$

$$\rightarrow s_1 = 3, s_2 = 2$$

The Hard Problem: Learning With Errors (LWE)

What happens if we add a little noise?

If we intentionally add a tiny, random **error (noise)** to the end of each equation, the math completely "breaks".

Learning With Errors (LWE)

$$15 = 4s_1 + 3s_2 + e_1$$

$$8 = 2s_1 + 1s_2 + e_2$$

$$\rightarrow s_1 = ?, s_2 = ?$$

Even quantum computers struggle to find $\vec{s}$ when noise is present!

LWE Encryption: Step 1 - The Ingredients

How do we actually encrypt a number?

Before doing any math, we must gather our four cryptographic components.

☰ System Setup & Ingredients

- **Secret Key ($\vec{s}$):** The master key used to both encrypt and decrypt the message.
- **Public Mask ($\vec{a}$):** A new, random vector generated entirely fresh *for every single encryption*.
- **Noise (e):** The crucial tiny error that makes the system of equations impossible to solve.
- **Scaling Factor (Δ):** A multiplier that shifts our message to the left, protecting it from being damaged by the noise.

Note: The vectors $\vec{s}$ and $\vec{a}$ must be the exact same length!

LWE Encryption: Step 2 - Preparing the Payload

Formatting the Data Box

We do not encrypt the raw message m directly. First, we must construct the "Payload" (the data box we visualized earlier) by scaling the message and injecting the noise.

Payload Construction

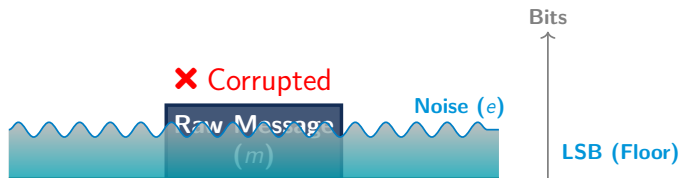
$$\text{Payload} = \Delta m + e$$

- **Multiply by Δ :** Shifts the message m to the upper, significant bits.
- **Add Noise e :** Fills the empty lower bits with random Gaussian noise to guarantee security against attacks.

Intuition: Without Scaling Factor (Disaster)

Leaving the data on the floor

Imagine our cryptographic space as a room. The random noise (e) we must add is like **floodwater** creeping across the floor. If we leave our valuable message (m) directly on the floor without scaling, it mixes with the noise.

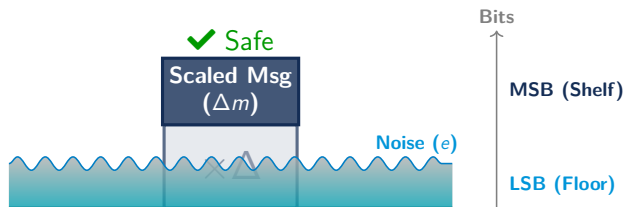


→ Result: The message and noise overlap in the LSB. Irreversible decryption failure!

Intuition: With Scaling Factor (Safe)

Building a high shelf (Δ)

Multiplying m by the scaling factor Δ is like **building a high shelf**. We safely lift the message to the top shelf, leaving the floodwater harmlessly on the floor.

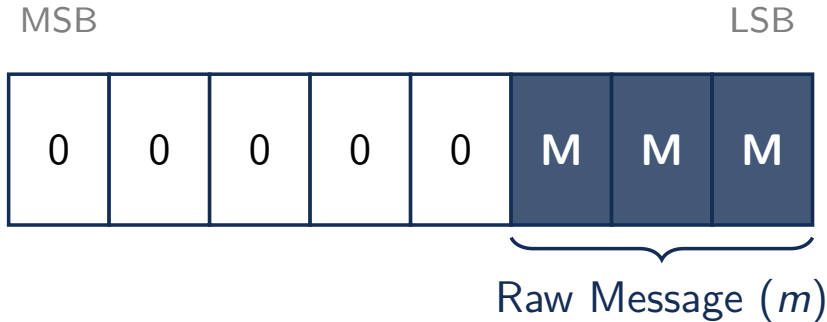


The Payload:

$$\underbrace{\Delta m}_{\text{Top Shelf (MSB)}} + \underbrace{e}_{\text{Floodwater (LSB)}}$$

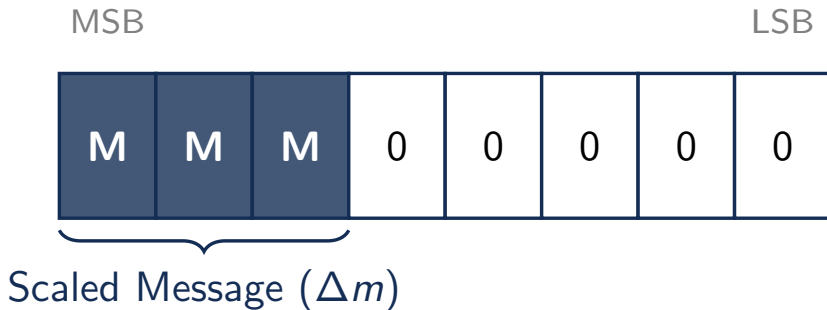
Visualizing the "Data Box" (1/3)

Step 1: Raw Message. We start with the raw message (m) in the least significant bits (LSB).



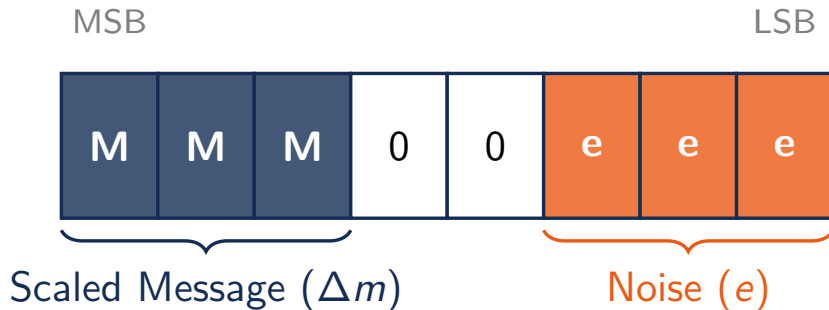
Visualizing the "Data Box" (2/3)

Step 2: Scaling. Multiplying by Δ shifts the message to the most significant bits (MSB).



Visualizing the "Data Box" (3/3)

Step 3: Adding Noise. We safely hide the random noise (e) in the empty lower bits.



LWE Encryption: Phase 1 - The Ingredients

Gathering the Components

Since we already prepared our **Payload** ($\Delta m + e$) in the previous step, we now only need three main components on the table:



New Mask ($\vec{a}$)

A completely fresh, random public vector.



Secret Key ($\vec{s}$)

Our static master key.



Prepared Payload

The data box ($\Delta m + e$) we just built.

LWE Encryption: Phase 2 - The Calculation

Locking the Payload

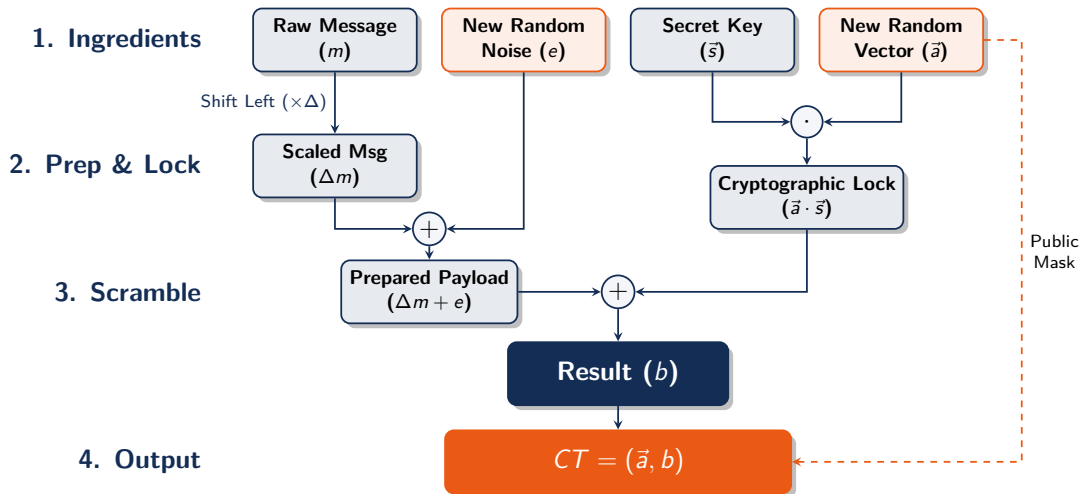
Now we mathematically add the prepared **Payload** to our cryptographic lock (the dot product) to completely scramble the data:

The Encryption Equation

$$b = \underbrace{(\vec{a} \cdot \vec{s})}_{\text{Lock}} + \underbrace{\text{Payload}}_{\Delta m + e} \pmod{q}$$

The Final Ciphertext Pair: $CT = (\vec{a}, b)$

LWE Encryption: Visualizing the Flow



LWE Decryption: Step 1 - Removing the Mask

How do we get our message back?

To decrypt the ciphertext $CT = (\vec{a}, b)$, the receiver uses their Secret Key $(\vec{s})$ to reverse the encryption. The first step is to remove the cryptographic lock.

Step 1: Remove the Mask

$$b - (\vec{a} \cdot \vec{s}) = \Delta m + e$$

By mathematically subtracting the mask $(\vec{a} \cdot \vec{s})$, we successfully unlock the ciphertext. We are now left with the "Data Box" containing our scaled message and the noise.

LWE Decryption: Step 2 - Extracting the Message

Destroying the Noise

Now that we have the exposed payload $(\Delta m + e)$, we need to extract the raw message m without the noise.

✓ Step 2: Scale Down and Round

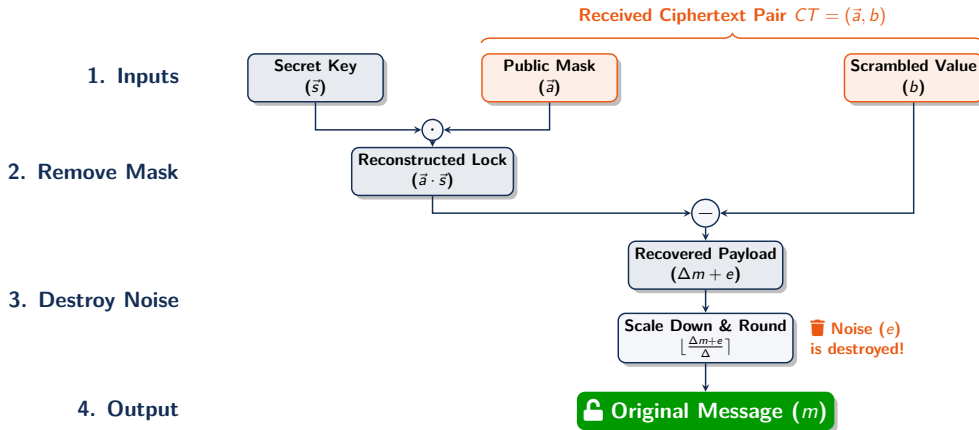
$$\left\lfloor \frac{\Delta m + e}{\Delta} \right\rfloor = m$$

The Magic of Rounding:

Dividing by Δ and rounding to the nearest integer completely destroys the noise e , restoring the perfect message m !

(This works flawlessly, provided the noise is small enough: $e < \Delta/2$)

LWE Decryption: Visualizing the Flow



Homomorphic Addition: Step 1 - Two Messages

Setting up the scenario

Let's encrypt two different messages, m_1 and m_2 , using the exact same Secret Key ($\vec{s}$). Notice that each gets a completely new mask and new noise!

Ciphertext 1

$$CT_1 = (\vec{a}_1, b_1) \quad \text{where} \quad b_1 = (\vec{a}_1 \cdot \vec{s}) + \Delta m_1 + e_1$$

Ciphertext 2

$$CT_2 = (\vec{a}_2, b_2) \quad \text{where} \quad b_2 = (\vec{a}_2 \cdot \vec{s}) + \Delta m_2 + e_2$$

Homomorphic Addition: Step 2 - Cloud Computation

Adding blindly in the cloud

The server does not know $\vec{s}$, m_1 , or m_2 . It simply adds the public vectors and the scalars together.

 Computing CT_{SUM}

$$CT_{SUM} = (\vec{a}_1 + \vec{a}_2, b_1 + b_2)$$

What happened inside the new scalar?

If we mathematically expand $b_1 + b_2$, the math perfectly aligns:

$$b_{SUM} = (\vec{a}_1 + \vec{a}_2) \cdot \vec{s} + \Delta(m_1 + m_2) + (e_1 + e_2)$$

Homomorphic Addition: Step 3 - The Magic of Decryption

Unlocking the sum

The receiver downloads CT_{SUM} and uses their Secret Key ($\vec{s}$) to decrypt it exactly like a normal ciphertext!

1. Remove the combined mask

$$b_{SUM} - (\vec{a}_1 + \vec{a}_2) \cdot \vec{s} = \Delta(m_1 + m_2) + (e_1 + e_2)$$

2. Scale down and round

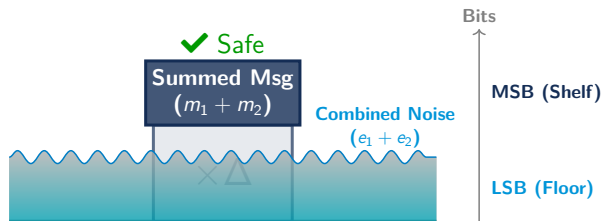
$$\left\lfloor \frac{\Delta(m_1 + m_2) + (e_1 + e_2)}{\Delta} \right\rfloor = m_1 + m_2$$

The messages added up cleanly, and the noises just gently added together!

Intuition: Homomorphic Addition (Safe)

The floodwater rises, but the shelf is high

When we homomorphically add two ciphertexts, the messages add up safely on the top shelf ($m_1 + m_2$). The random noises also add up ($e_1 + e_2$), causing the floodwater to rise. However, it is still far from reaching the data!



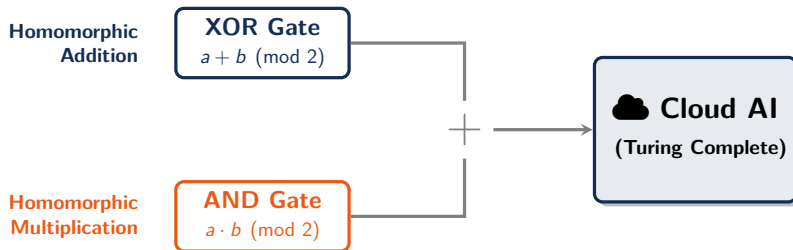
The Payload:

$$\underbrace{\Delta(m_1 + m_2)}_{\text{Top Shelf (MSB)}} + \underbrace{(e_1 + e_2)}_{\text{Floodwater (LSB)}}$$

The Missing Piece: Turing Completeness

Why do we need both operations?

Partial homomorphism is not enough. To run arbitrary software or evaluate AI models, our system must be **Turing Complete**. In binary logic, every possible program can be built using just two basic operations:



→ We successfully conquered addition. Now, we must face the ultimate challenge: Multiplication!

Homomorphic Multiplication: Step 1 - Two Messages

Setting up the scenario

Just like with addition, we start with two completely different messages, m_1 and m_2 , encrypted under the same Secret Key ($\vec{s}$).

Ciphertext 1

$$CT_1 = (\vec{a}_1, b_1) \quad \text{where} \quad b_1 = (\vec{a}_1 \cdot \vec{s}) + \Delta m_1 + e_1$$

Ciphertext 2

$$CT_2 = (\vec{a}_2, b_2) \quad \text{where} \quad b_2 = (\vec{a}_2 \cdot \vec{s}) + \Delta m_2 + e_2$$

Homomorphic Multiplication: Step 2 - Cloud Computation

Multiplying in the cloud

The server multiplies the ciphertexts. Unlike simple addition, this requires complex mathematical operations, but let's focus purely on what happens inside the "Data Box" (the payload).

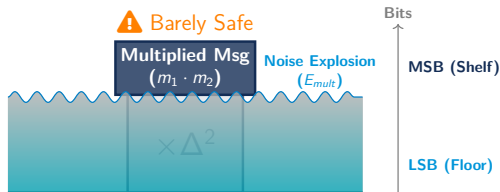
✕ Multiplying the Payloads

$$\begin{aligned} & (\Delta m_1 + e_1) \cdot (\Delta m_2 + e_2) \\ &= \Delta^2(m_1 \cdot m_2) + \underbrace{\Delta(m_1 e_2 + m_2 e_1) + e_1 e_2}_{E_{mult}} \end{aligned}$$

Intuition: Homomorphic Multiplication (Danger)

The "Noise Explosion" tsunami

When we multiply ciphertexts, the cross-terms multiply the noise by the massive scaling factor Δ . The floodwater rapidly surges into a **tsunami**. It is much higher than in addition and almost reaches the data! It is **barely safe** now, but **one more multiplication** will definitely cause an overflow and irreversibly destroy the message.



The Payload:

$$\underbrace{\Delta^2(m_1 m_2)}_{\text{Top Shelf (MSB)}} + \underbrace{\Delta(m_1 e_2 + m_2 e_1) + e_1 e_2}_{\text{Tsunami } (E_{mult})}$$

Homomorphic Multiplication: Key Observation

What just happened to our data?

$$\text{Result} = \Delta^2(m_1 \cdot m_2) + E_{\text{mult}}$$

Q Key Observation

The messages are successfully multiplied, but notice two critical side effects:

- **Squared Scaling Factor:** Our multiplier grew from Δ to Δ^2 .
- **Noise Explosion:** The cross-terms merged into a massive new combined noise (E_{mult}).

Homomorphic Multiplication: Step 3 - The Magic of Decryption

Unlocking the product

The receiver downloads the multiplied ciphertext and uses their Secret Key to remove the complex mask. They are left with the multiplied payload!

1. Remove the combined mask

$$\text{Payload} = \Delta^2(m_1 \cdot m_2) + E_{mult}$$

2. Scale down and round

$$\left\lfloor \frac{\Delta^2(m_1 \cdot m_2) + E_{mult}}{\Delta^2} \right\rfloor = m_1 \cdot m_2$$

Notice that we must now divide by Δ^2 to successfully recover the raw message!

The Decryption Failure Threshold

Is infinite computation impossible?

If we keep multiplying, the noise e grows exponentially. Eventually, it becomes so massive that it spills over the lower bits and **overwrites the actual message**.

☠ The Point of No Return

If the noise exceeds our safety margin ($e \geq \Delta/2$), the data is irreversibly corrupted. Decryption will yield pure garbage!

The Solution: Defeating the Noise

Are we doomed to only a few multiplications?

No! Cryptographers invented a magic trick called **Bootstrapping**.

💡 Bootstrapping (Part 3)

A brilliant mechanism to "refresh" a ciphertext and wash away the noise mid-computation, without ever exposing the underlying plaintext.

Enough Theory... Time for Practice!

We have survived the mathematical storm!

We've covered the LWE problem, the noise agony, and the magic of Bootstrapping. In the **next part (Part 3)**, we will leave the theory behind and meet a truly powerful tool:

</> The OpenFHE Library

- **Industry Standard:** A prominent, open-source library implementing the Homomorphic Encryption Standard.
- **Unified API:** It supports the major FHE schemes under one roof:
 - **TFHE:** Lightning-fast Boolean logic gates.
 - **BFV:** Exact integer arithmetic and vector batching.
 - **CKKS:** Approximate floating-point numbers for AI.
- **Python Wrapper:** We will use `openfhe-python` to quickly prototype our encrypted cloud circuits.